IIC2523
Sistemas Distribuidos

Hernán F. Valdivieso López
(2026 - 1 / Clase 03)

Temas de la clase

1. Comunicación de red
    a. Tipos y acoplamientos
    b. Paradigmas
2. Invocación remota
    a. Características generales
    b. *Remote Procedure Call* (RPC)
    c. RPC en Python
    d. gRPC y *protocol buffer*
3. Comunicación directa (solo si alcanzamos, sino lo vemos en la clase 4)
    a. Características y *Socket*
    b. *Socket* en Python con TCP/IP

Comunicación entre nodos I
Características y Comunicación remota

Comunicación de red

Tipos y acoplamientos

Paradigma

Comunicación de red - Tipos

- El Intercambio de información entre dispositivos conectados a una red se puede clasificar según dos criterios:

Persistencia de los mensajes y sincronía de los mensajes

Comunicación de red - Tipos

- El Intercambio de información entre dispositivos conectados a una red se puede clasificar según dos criterios:

Persistencia de los mensajes y sincronía de los mensajes

- **Transitoria**: el mensaje vive sólo mientras las aplicaciones del emisor y receptor están en ejecución. Si uno de los 2 entes cae, el mensaje se pierde.
  - Llamada de *Google Meet* (no grabada), mensajes por `sockets` de Python

Comunicación de red - Tipos

- El Intercambio de información entre dispositivos conectados a una red se puede clasificar según dos criterios:

Persistencia de los mensajes y sincronía de los mensajes

- **Transitoria**: el mensaje vive sólo mientras las aplicaciones del emisor y receptor están en ejecución. Si uno de los 2 entes cae, el mensaje se pierde.
  - Llamada de *Google Meet* (no grabada), mensajes por `sockets` de Python
- **Persistente**: se almacena, en algún lado, el mensaje enviado por el emisor el tiempo que tome entregarlo al receptor.
  - *Outlook*, *Telegram*.

Comunicación de red - Tipos

- El Intercambio de información entre dispositivos conectados a una red se puede clasificar según dos criterios:

Persistencia de los mensajes y **sincronía de los mensajes**

- **Síncrona**: El emisor queda bloqueado hasta que confirme que su mensaje es recibido, e incluso que se tenga respuesta del mensaje.
  - Acceder a una páginas web, la librería "`requests`" de Python.

Comunicación de red - Tipos

- El Intercambio de información entre dispositivos conectados a una red se puede clasificar según dos criterios:

Persistencia de los mensajes y **sincronía de los mensajes**

- **Síncrona**: El emisor queda bloqueado hasta que confirme que su mensaje es recibido, e incluso que se tenga respuesta del mensaje.
  - Acceder a una páginas web, la librería "`requests`" de Python.
- **Asíncrona**: El emisor puede continuar ejecutando inmediatamente después de enviar el mensaje, incluso si el mensaje todavía no llega al receptor.
  - Gmail, transmisión por Zoom.

Comunicación de red - Acoplamiento

- Con los conceptos anteriores, una comunicación se puede caracterizar según el tipo de acoplamiento que tiene para que sea exitosa:

Acoplamiento Temporal o Acoplamiento Espacial (o Referencial)

Comunicación de red - Acoplamiento

- Con los conceptos anteriores, una comunicación se puede caracterizar según el tipo de acoplamiento que tiene para que sea exitosa:

Acoplamiento Temporal o Acoplamiento Espacial (o Referencial)

- **Acoplado**: El emisor y el receptor deben existir al mismo tiempo para comunicarse.
  - Llamada por Zoom, transmisión por Youtube.

Comunicación de red - Acoplamiento

- Con los conceptos anteriores, una comunicación se puede caracterizar según el tipo de acoplamiento que tiene para que sea exitosa:

Acoplamiento Temporal o Acoplamiento Espacial (o Referencial)

- **Acoplado**: El emisor y el receptor deben existir al mismo tiempo para comunicarse.
  - Llamada por Zoom, transmisión por Youtube.
- **Desacoplado**: El emisor y receptor pueden existir en tiempos distintos.
  - Memoria compartida, las aplicaciones de mensajería como gmail.

Comunicación de red - Acoplamiento

- Con los conceptos anteriores, una comunicación se puede caracterizar según el tipo de acoplamiento que tiene para que sea exitosa:

Acoplamiento Temporal o **Acoplamiento Espacial (o Referencial)**

- **Acoplado**: El emisor conoce o necesita conocer la identidad de los receptores.
  - Mensaje privado de Whatsapp, Correo electrónico.

Comunicación de red - Acoplamiento

- Con los conceptos anteriores, una comunicación se puede caracterizar según el tipo de acoplamiento que tiene para que sea exitosa:

Acoplamiento Temporal o **Acoplamiento Espacial (o Referencial)**

- **Acoplado**: El emisor conoce o necesita conocer la identidad de los receptores.
  - Mensaje privado de Whatsapp, Correo electrónico.
- **Desacoplado**: El emisor no necesita conocer la identidad del receptor o receptores. La comunicación se realiza a través de una entidad intermediaria.
  - Publicación en *instagram*, Aviso de Canvas.

Comunicación de red - Paradigmas

- Se pueden distinguir 3 paradigmas de comunicación no excluyentes:

Directa

La forma más simple de comunicación entre procesos a través del paso de mensajes.

Ligero, eficiente y minimalista.

Hay acoplamiento referencial.

Indirecta

La comunicación se realiza **a través de un intermediario distinto al SSOO.**

Sin acoplamiento directo entre el emisor y el receptor.

Según cómo sea el intermediario, puede que los mensajes sean persistentes o transitorios.

Invocación Remota

Se invocan operaciones o métodos que se ejecutan en otra máquina conectada en red.

Generalmente, se encapsula el paso de mensaje para simplificar la comunicación.

Fuente: Distributed Systems: Concepts and Design (5ª ed.), Coulouris, G., Dollimore, J., & Kindberg, T. (2005)

Invocación remota

Características generales

Remote Procedure Call (RPC)

RPC en Python

gRPC y *protocol buffer*

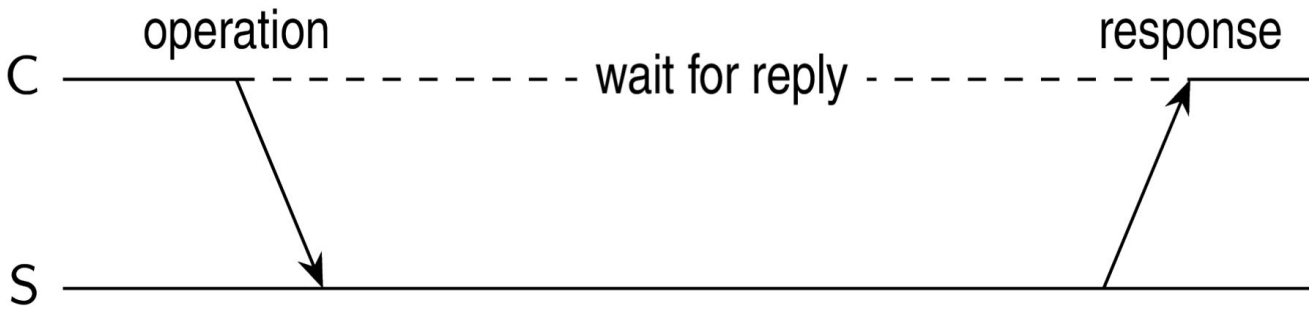
Invocación Remota - Características generales

- Paradigma de comunicación donde se invocan operaciones o métodos que se ejecutan en otra máquina/proceso.

Fuente: Distributed Systems: Concepts and Design (5ª ed.), Coulouris, G., Dollimore, J., & Kindberg, T. (2005)

Invocación Remota - Características generales

- Paradigma de comunicación donde se invocan operaciones o métodos que se ejecutan en otra máquina/proceso.
- Por defecto, este paradigma utiliza un protocolo del tipo *request-reply*.
  - Enviar un mensaje y esperar respuesta.
  - Protocolo síncrono y acoplado temporalmente.
  - Es posible hacer este protocolo asíncrono con otras herramientas como *threads*, promesas, etc.).



Fuente: Distributed Systems: Concepts and Design (5ª ed.), Coulouris, G., Dollimore, J., & Kindberg, T. (2005)

Invocación Remota - Características generales

- Existen 2 enfoques muy conocidos: REST y RPC.

Invocación Remota - REST

- Existen 2 enfoques muy conocidos: **REST** y RPC.
  - ***Re****presentational* ***S****tate* ***T****ransfer*.
  - Utiliza protocolo HTTP y se alínea a sus estándares para comunicarse.
  - Uso de URLs para identificar los distintos recursos/*endpoint* y las operaciones HTTP (GET, POST, etc.) para manipularlos.
  - Cada ejecución es ***Stateless*** → no tiene memoria de las ejecuciones anteriores.
  - Sus mensajes son *human readable*.

```
GET /anime/detective-conan HTTP/1.1
Host: www3.animeflv.net
Remote Address: 185.178.208.140:443
Accept: text/html
```

Invocación Remota - REST

- Existen 2 enfoques muy conocidos: **REST** y RPC.
  - ***Re**presentational **S**tate **T**ransfer*.
  - Utilizan protocolo HTTP, por lo que también está [illegible]
  - Sus mensajes son *human readable*.

Al momento de definir el concepto de "Invocación remota" (tipos años 70 👴), no existía REST y por lo mismo la bibliografía no la incluye, pero sus características permiten que sea considerada dentro de este paradigma.

```
GET /anime/detective-conan HTTP/1.1
Host: www3.animeflv.net
Remote Address: 185.178.208.140:443
Accept: text/html
```

Invocación Remota - *Remote Procedure Call* (RPC)

- Enfoque donde se busca la **transparencia de acceso y de ubicación**
  - Se llaman a las funciones/procedimientos de una **forma idéntica a como se llamaría de forma local**.

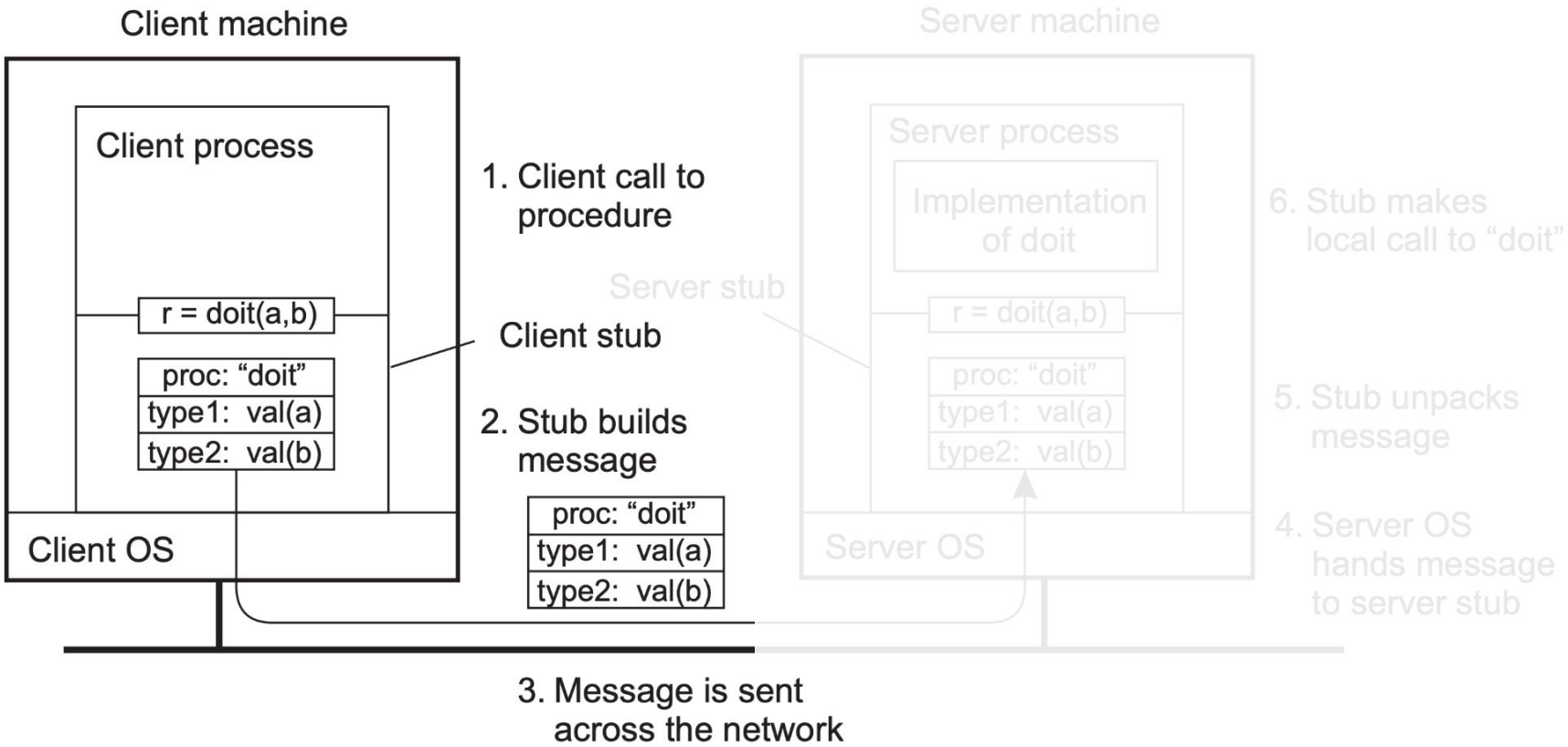
Invocación Remota - *Remote Procedure Call* (RPC)

- Enfoque donde se busca la **transparencia de acceso y de ubicación**
  - Se llaman a las funciones/procedimientos de una **forma idéntica a como se llamaría de forma local**.
- Se define un nuevo componente: *stub*
  - ***Client stub***: convierte los llamados del cliente y sus argumentos en mensajes para enviar al servidor. Luego interpreta el mensaje obtenido y la convierte en el resultado de la función ejecutada.
  - ***Server stub***: interpreta los mensajes del cliente para identificar qué función ejecutar y luego genera el mensaje con la respuesta de dicha ejecución.

Invocación Remota - *Remote Procedure Call* (RPC)

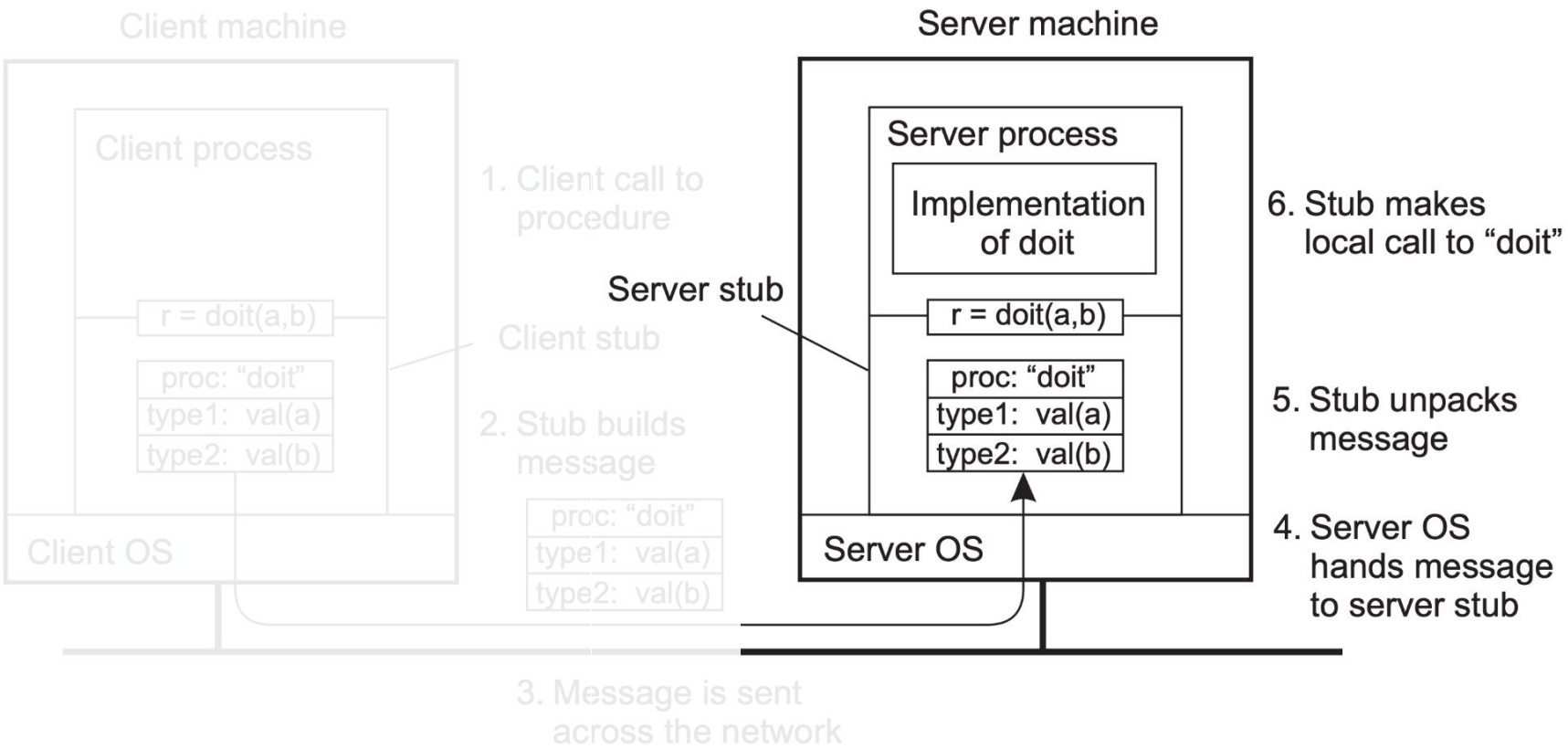
- Enfoque donde se busca la **transparencia de acceso y de ubicación**
  - Se llaman a las funciones/procedimientos de una **forma idéntica a como se llamaría de forma local**.
- Se define un nuevo componente: *stub*
  - ***Client stub***: convierte los llamados del cliente y sus argumentos en mensajes para enviar al servidor. Luego interpreta el mensaje obtenido y la convierte en el resultado de la función ejecutada.
  - ***Server stub***: interpreta los mensajes del cliente para identificar qué función ejecutar y luego genera el mensaje con la respuesta de dicha ejecución.
- La comunicación se realiza mediante serialización (frecuentemente binaria), aunque hoy en día existen implementaciones que usan formatos legibles por humanos.

Invocación Remota - *Remote Procedure Call* (RPC)



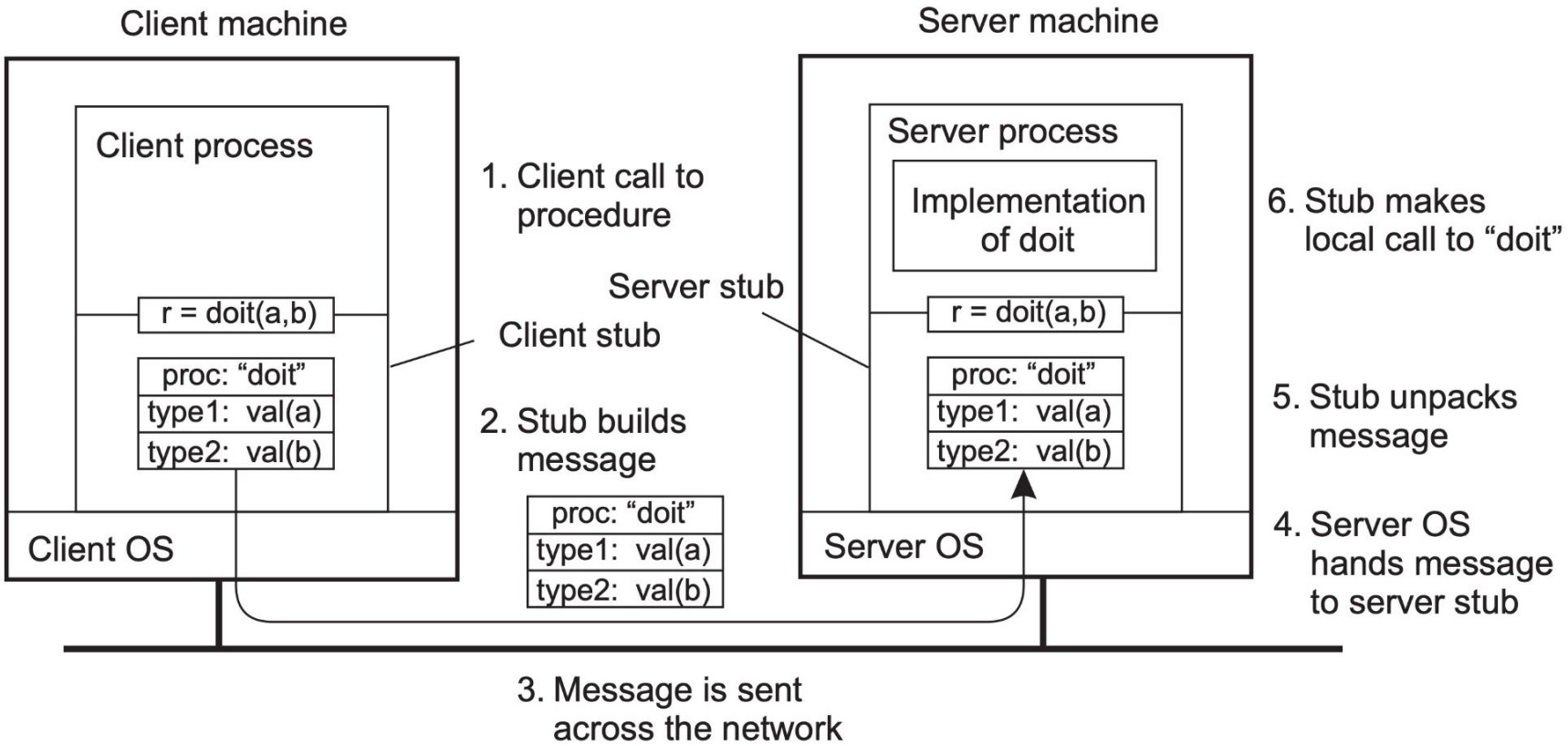
Fuente: Distributed Systems: Concepts and Design (5ª ed.), Coulouris, G., Dollimore, J., & Kindberg, T. (2005)

Invocación Remota - *Remote Procedure Call* (RPC)



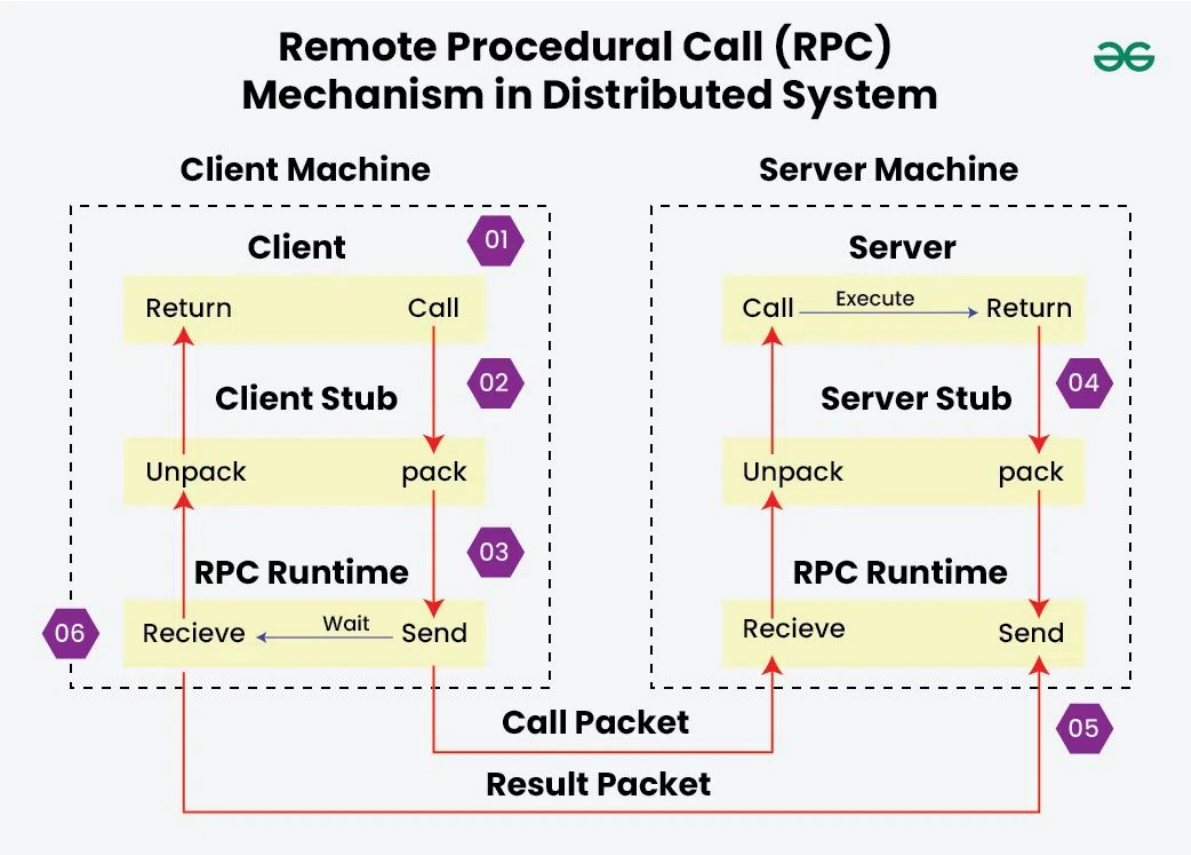
Fuente: Distributed Systems: Concepts and Design (5ª ed.), Coulouris, G., Dollimore, J., & Kindberg, T. (2005)

Invocación Remota - *Remote Procedure Call* (RPC)



Fuente: Distributed Systems: Concepts and Design (5ª ed.), Coulouris, G., Dollimore, J., & Kindberg, T. (2005)

Invocación Remota - *Remote Procedure Call* (RPC)



Fuente: [What is Remote Procedural Call (RPC) Mechanism in Distributed System?]

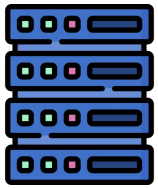
Invocación Remota - REST o RPC

GET /animes

```
[{"id": 1, "name": "Evangelion"},
 {"id": 2, "name": "Code Geass"} … ]
```

GET /animes/2

```
{"id": 2,
 "name": "Code Geass",
 "seasons": 2, … }
```



REST

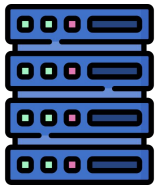
Invocación Remota - REST o RPC

GET /animes

[{"id": 1, …},
{"id": 2, …} …]

GET /animes/2

{"id": 2 … }



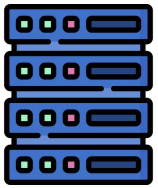
REST

s.animes()

[{"id": 1, …},
{"id": 2, …} …]

s.animes(id=2)

{"id": 2 … }



RPC

Invocación Remota - REST o RPC

- Al momento de comunicarse, los *stubs* pueden usar comunicación directa mediante *sockets* y sus propias normas, o emplear otros protocolos como HTTP. No obstante, no necesariamente respetan los estándares de HTTP:
  - El servidor RPC puede mantener estado y recordar ejecuciones pasadas.
  - Un GET puede modificar información.
  - Un POST puede ser solo para leer datos.
- Se puede incluir un intermediario (comunicación indirecta) para ofrecer funcionalidades adicionales.

Invocación Remota - RPC (servidor)

```
DATA = []

def f1(x, y):
    return x + y

def f2():
    return "Hello World"

def f3(x):
    DATA.append(x)
    return DATA[0]
```

Invocación Remota - RPC (servidor)

```python
DATA = []

def f1(x, y):
    return x + y

def f2():
    return "Hello World"

def f3(x):
    DATA.append(x)
    return DATA[0]
```

```python
from xmlrpc.server import SimpleXMLRPCServer


ip_port = ("localhost", 4444)
server = SimpleXMLRPCServer(ip_port, allow_none=True)
server.register_function(f1, "add")
server.register_function(f2, "hello_world")
server.register_function(f3, "append")
server.serve_forever()
```

Invocación Remota - RPC (cliente)

```python
from xmlrpc.client import ServerProxy

servidor = ServerProxy("http://localhost:4444/", allow_none=True)
print("3 + 8 =", servidor.add(3, 8))
print("-->", servidor.hello_world())
print("DATA: ", servidor.append("Probando"))
print("DATA: ", servidor.append("4444"))
print("Concatenar string: ", servidor.add("AN", "YA"))
print("DATA: ", servidor.append(None))
```

Invocación Remota - *Remote Method Invocation* (RMI)

- Contexto: Java es un lenguaje que se basa fuertemente en la Programación Orientado a Objetos.
- RMI fue diseñado para implementar RPC en Java.
- RMI es un tipo específico de RPC donde se invocan **métodos** de un objeto dentro del paradigma de OOP.

Invocación Remota - RMI (servidor)

```python
class RMIServer:
    def __init__(self):
        self.data = []

    def add(self, x, y):
        return x + y

    def hello_world(self):
        return "Hello World"

    def append(self, x):
        self.data.append(x)
        return self.data[0]
```

```python
from xmlrpc.server import SimpleXMLRPCServer


rmi = RMIServer()
ip_port = ("localhost", 4444)
server = SimpleXMLRPCServer(ip_port,
                            allow_none=True)
server.register_instance(rmi)
server.serve_forever()
```

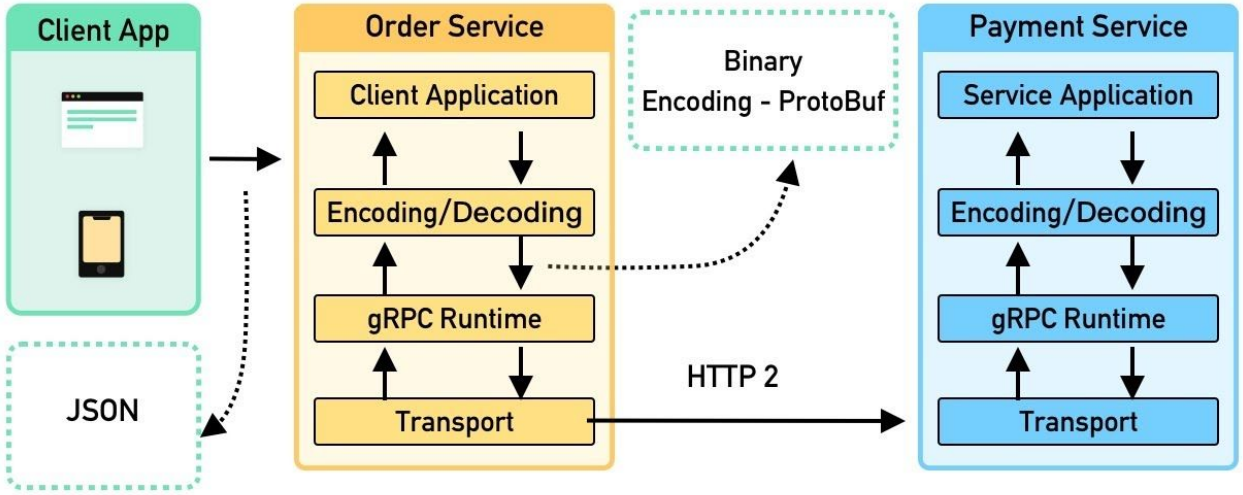
Invocación Remota - RMI (cliente)

```python
# Se mantiene igual aunque el servidor use register_instance o register_function
from xmlrpc.client import ServerProxy

servidor = ServerProxy("http://localhost:4444/", allow_none=True)
print("3 + 8 =", servidor.add(3, 8))
print("-->", servidor.hello_world())
print("DATA: ", servidor.append("Probando"))
print("DATA: ", servidor.append("4444"))
print("Concatenar string: ", servidor.add("AN", "YA"))
print("DATA: ", servidor.append(None))
```

Invocación Remota - gRPC

- Implementación específica de RPC por Google el 2015.
- Utiliza un protocolo de comunicación más rápido y eficiente (HTTP/2).
- Admite más formatos de serialización como *Protocol Buffers*, JSON y XML.



Fuente: What is RPC? gRPC Introduction.

Invocación Remota - gRPC

- Implementación específica de RPC por Google el 2015.
- Utiliza un protocolo de comunicación más rápido y eficiente (HTTP/2).
- Admite más formatos de serialización como *Protocol Buffers*, JSON y XML.
- ***Protocol Buffers***
    1. Se define la estructura de los datos a enviar en un archivo de esquema.
    2. *Protocol Buffers* genera un código, en el lenguaje deseado, con la estructura lista para importar y utilizar.
    3. Se instancian los datos usando dicha estructura.
    4. Se serializar y deserializar esas instancias en binario para su transmisión.

Invocación Remota - gRPC

- *Protocol Buffers*

```
# Esquema
syntax = "proto3";

message Character {
   string username = "";
   string phase = "";
}
```

```
import character_pb3 # Este codigo creado por pb

character = character_pb3.Character()
character.username = "Anya"
character.phase = "Waku Waku"

with open("data.anya", "wb") as file:
    file.write(character.SerializeToString())


character_new = character_pb3.Character()
with open("data.anya", "rb") as file:
    character_new.ParseFromString(file.read())
```

Comunicación directa

Características

Socket

Socket en Python con TCP/IP

Comunicación directa

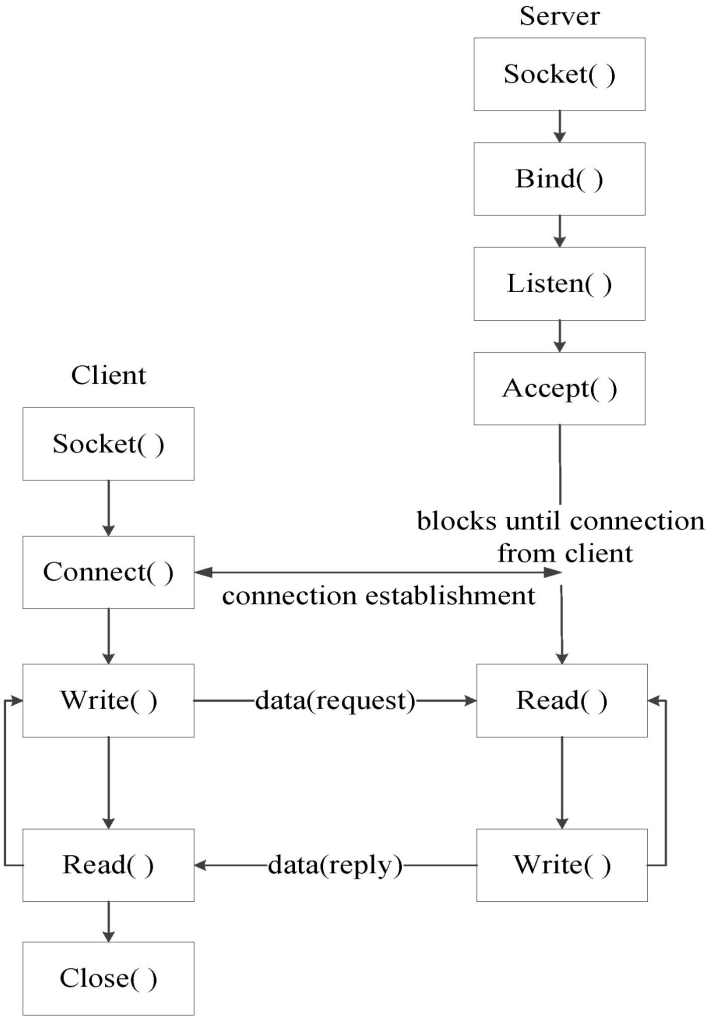
- La forma más simple de comunicación entre procesos es a través del pasaje de mensajes.
- El proceso emisor debe especificar el destino del proceso receptor (puede ser con una dirección y un puerto).

Comunicación directa

- La forma más simple de comunicación entre procesos es a través del pasaje de mensajes.
- El proceso emisor debe especificar el destino del proceso receptor (puede ser con una dirección y un puerto).
- Herramienta clásica: *Socket*
  - Es una interfaz de entrada-salida de datos que permite la intercomunicación entre procesos.
  - Se pueden asociar al protocolo UDP o TCP.
  - Se debe diferenciar si el socket actúa como cliente o como servidor.
  - Con el uso de *thread* es posible tener interesantes comportamientos como múltiples servidores en un mismo proceso, atender múltiples clientes, etc.

Comunicación directa - Socket

- Es una interfaz de entrada-salida de datos que permite la intercomunicación entre procesos.
- Se pueden asociar al protocolo UDP o TCP.
- Se debe diferenciar si el socket actúa como cliente o como servidor.
- Con el uso de *thread* es posible tener interesantes comportamientos como múltiples servidores en un mismo proceso, atender múltiples clientes, etc.



Socket - Python (Servidor)

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(('localhost', 4444))
sock.listen(5)
i = 0
while i < 2:
    connection, client_address = sock.accept()
    largo = connection.recv(4)
    mensaje = connection.recv(int.from_bytes(largo, byteorder="big"))
    print(mensaje.decode("UTF-8"))
    connection.sendall(largo + mensaje)
    connection.close()
    i += 1
sock.close()
```

Socket - Python (Cliente)

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(('localhost', 4444))

mensaje = "waku waku".encode("UTF-8")
largo = len(mensaje).to_bytes(4, byteorder="big")
sock.sendall(largo)
sock.sendall(mensaje)
largo = sock.recv(4)
data = sock.recv(int.from_bytes(largo, byteorder="big"))
print(data.decode("UTF-8"))
sock.close()
```

Poniendo a prueba lo que hemos aprendido 👀

Supongamos que su profesor hará una conferencia por Zoom para hablar de anime. Pero, como no tiene Zoom Pro, no va a grabar esa conferencia. Además, para mantener la privacidad de usuarios, Zoom será un intermediario que recibirá la información y reenviará la transmisión a todos los presentes sin que el profesor sepa realmente sus identidades.

¿Cuál de las siguientes afirmaciones describe de forma más precisa la naturaleza del intercambio de información?

a) Es una comunicación persistente, asíncrona y acoplado referencialmente.

b) Es una comunicación transitoria, síncrona y acoplado referencialmente.

c) Es una comunicación transitoria, asíncrona y acoplada temporal.

d) Es una comunicación desacoplada temporal y referencialmente.

e) Es una comunicación persistente, síncrona y acoplado referencialmente.

Próximos eventos

Próxima clase

- Comunicación entre nodos II
  - Comunicación indirecta
  - Algunos protocolos de comunicación

Evaluación

- Actividad **obligatoria**: control de 2 preguntas sobre la clase de hoy.

IIC2523
Sistemas Distribuidos

Hernán F. Valdivieso López
(2026 - 1 / Clase 03)

Créditos (animes utilizados)

K-On!

Boku no Hero Academia